Data Structures and Algorithm

Xiaoqing Zheng
zhengxq@fudan.edu.cn







Solving systems of linear equations

Matrix-vector equation

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

We can equivalently write as $Ax = b$.

If A is ***nonsingular***, it possesses an inverse A^{-1}, and

$$x = A^{-1}b$$

It suffers in practice from numerical instability.

LUP decomposition

The idea behind LUP decomposition is to find three $n \times n$ matrices L, U, and P such that

$$PA = LU$$

where

- L is a unit lower-triangular matrix,
- U is an upper-triangular matrix, and
- P is a permutation matrix.

LUP decomposition example

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{P}\underbrace{\begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix}}_{A} =$$

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 2 & 1 & 7 & 1 \end{pmatrix}}_{L}\underbrace{\begin{pmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix}}_{U}$$

Permutation matrix

$$\begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix} =$$

$$\begin{pmatrix} 2 & 19 & 10 & 23 \\ 6 & 13 & 5 & 19 \\ 2 & 3 & 1 & 5 \\ 4 & 10 & 11 & 31 \end{pmatrix}$$

Solving linear systems by LUP

Once we have found an LUP decomposition for A, we can solve equation, $Ax = b$ as follows.

Multiplying both sides of $Ax = b$ by P yields $PAx = Pb$.

$$\left.\begin{array}{l} PAx = Pb \\ PA = LU \end{array}\right\} \Longrightarrow LUx = Pb$$

Letting $y = Ux$, where x is the desired solution vector.

First, we solve the lower-triangular system by a method called "***forward substitution***."

$$Ly = Pb$$

Having solved for y, we then solve the upper-triangular system by a method called "***back substitution***."

$$Ux = y$$

Forward substitution

Given L, P, and b. For convenience, we represent the permutation P compactly by an array $\pi[1..n]$. For $i = 1, 2, \ldots, n$, the entry $\pi[i]$ indicates that $P_{i,\pi[i]} = 1$ and $P_{ij} = 0$ for $j \neq \pi[i]$. Since L is unit lower-triangular, we can rewrite equation as

$$
\begin{aligned}
y_1 &= b_{\pi[1]}, \\
l_{21}y_1 + y_2 &= b_{\pi[2]}, \\
l_{31}y_1 + l_{32}y_2 + y_3 &= b_{\pi[3]}. \\
&\vdots \\
l_{n1}y_1 + l_{n2}y_2 + l_{n3}y_3 + \cdots + y_n &= b_{\pi[n]}.
\end{aligned}
$$

Forward and back substitution

In general, we substitute $y_1, y_2, \ldots, y_{i-1}$ "***forward***" into the ith equation to solve for y_i.

$$y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j$$

Having solved for y, we solve for x using back substitution, which is similar to forward substitution. Here, we solve the nth equation first and work back to the first equation.

$$x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) / u_{ii}$$

Algorithm of solving linear systems

LUP-SOLVE(L, U, π, b)

1 $n = L.rows$

2 let x be a new vector of length n

3 **for** $i = 1$ **to** n

4 $\quad y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j$

5 **for** $i = n$ **downto** 1

6 $\quad x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) / u_{ii}$

7 **return** x

$$\Theta(n^2)$$

Computing an LU decomposition

We start with the case in which A is an $n \times n$ nonsingular matrix and P is absent (or, equivalently, $P = I_n$). In this case, we factor $A = LU$.

We use a process known as ***Gaussian elimination*** to create an LU decomposition. The algorithm to implement this method is ***recursive***. If $n = 1$, then we are done. For $n > 1$, we break A into four parts:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ v & A' \end{pmatrix}$$

Schur complement

Then, using matrix algebra (verify the equations simply multiplying through), we can factor A as

$$A = \begin{pmatrix} a_{11} & w^{\mathbf{T}} \\ v & A' \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ v / a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathbf{T}} \\ 0 & A' - vw^{\mathbf{T}} / a_{11} \end{pmatrix}$$

where $A' - vw^{\mathbf{T}} / a_{11}$ is called the ***Schur complement*** of A with respect to a_{11}. Clearly, if A is nonsingular, then the Schur complement is nonsingular.

Recursively find an LU decomposition

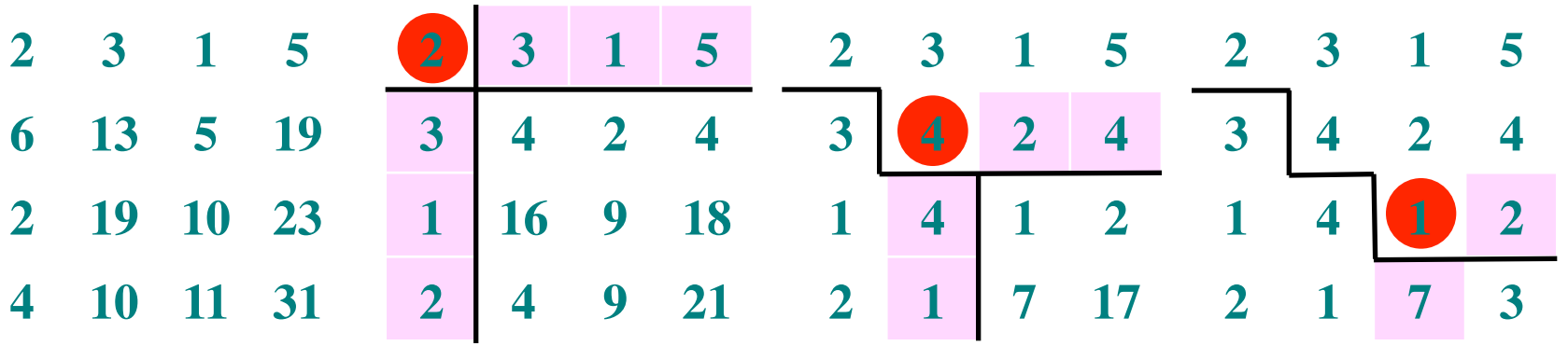
Because the Schur complement is nonsingular, we can recursively find an LU decomposition for it. Letting

$$A' - vw^{\mathrm{T}} / a_{11} = L'U'$$

where L' is unit lower-triangular and U' is upper-triangular. Then, we have

$$\begin{aligned} A &= \begin{pmatrix} 1 & 0 \\ v / a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & L'U' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v / a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^{\mathrm{T}} \\ 0 & U' \end{pmatrix} \\ &= LU \end{aligned}$$

LU decomposition example



$$\begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 2 & 1 & 7 & 1 \end{pmatrix} \begin{pmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

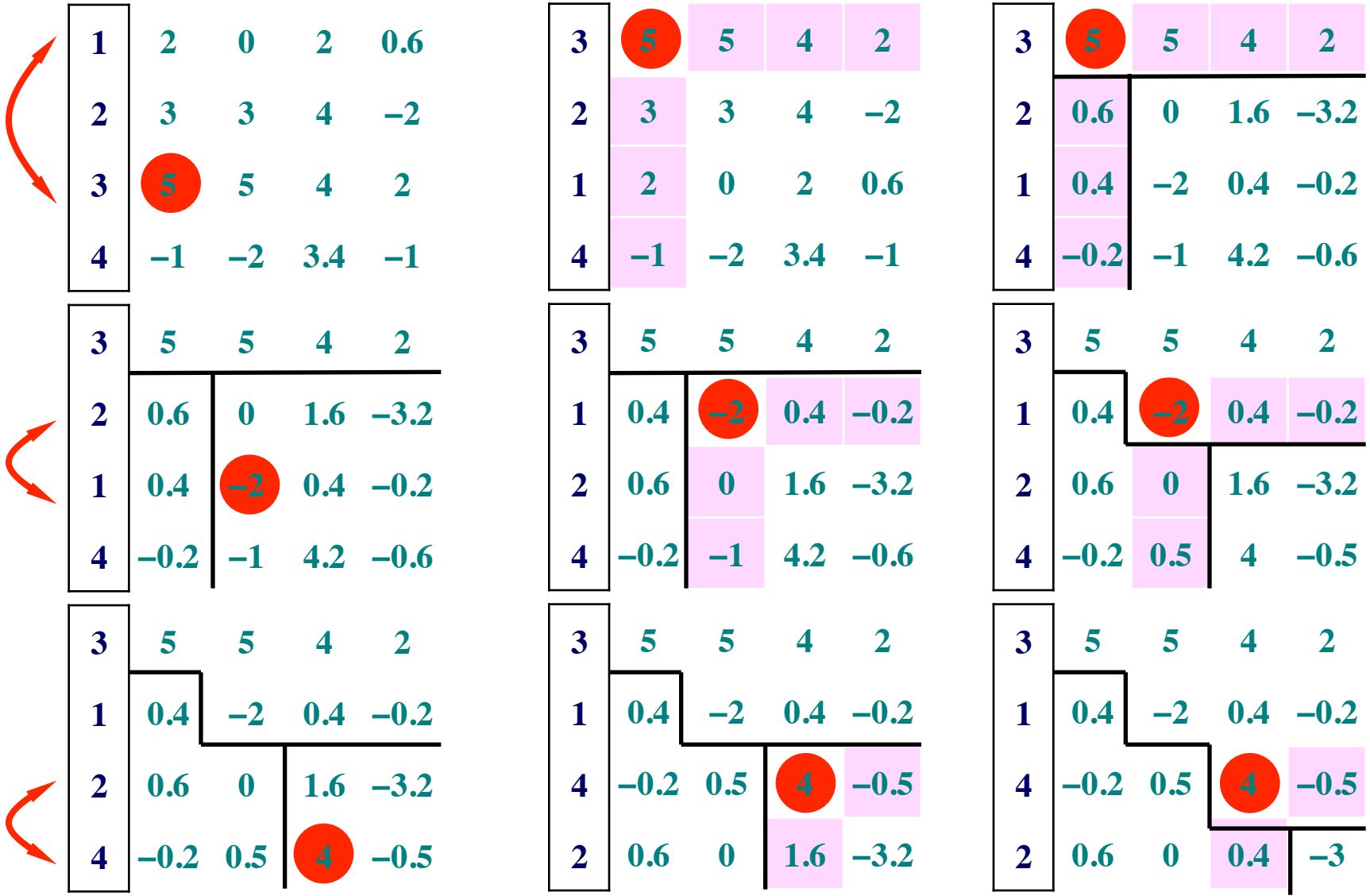
A L U

LU-decomposition algorithm

LU-DECOMPOSITION(A)

1 $n = A.rows$
2 **for** $k = 1$ **to** n
3 **for** $i = k + 1$ **to** n
4 $a_{ik} = a_{ik} / a_{kk}$
5 **for** $j = k + 1$ **to** n
6 $a_{ij} = a_{ij} - a_{ik}\, a_{kj}$

LUP decomposition example



LUP decomposition example

$$\begin{matrix} 2 & 0 & 2 & 0.6 \\ 3 & 3 & 4 & -2 \\ 5 & 5 & 4 & 2 \\ -1 & -2 & 3.4 & -1 \end{matrix} \Rightarrow \begin{matrix} \boxed{3} & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 4 & -0.2 & 0.5 & 4 & -0.5 \\ 2 & 0.6 & 0 & 0.4 & -3 \end{matrix}$$

$$\underset{P}{\begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}} \underset{A}{\begin{pmatrix} 2 & 0 & 2 & 0.6 \\ 3 & 3 & 4 & -2 \\ 5 & 5 & 4 & 2 \\ -1 & -2 & 3.4 & -1 \end{pmatrix}} = \underset{L}{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0.4 & 1 & 0 & 0 \\ -0.2 & 0.5 & 1 & 0 \\ 0.6 & 0 & 0.4 & 1 \end{pmatrix}} \underset{U}{\begin{pmatrix} 5 & 5 & 4 & 2 \\ 0 & -2 & 0.4 & -0.2 \\ 0 & 0 & 4 & -0.5 \\ 0 & 0 & 0 & -3 \end{pmatrix}}$$

LUD-decomposition algorithm

```
LUD-DECOMPOSITION(A)
1     n = A.rows
2     let π[1..n] be a new array
3     for i = 1 to n
4           π[i] = i
5     for k = 1 to n
6           p = 0
7           for i = k to n
8                 if |a_ik| > p
9                       p = |a_ik|
10                      k' = i
11          if p = = 0
12                error "singular matrix"
13          exchange π[k] with π[k']
14          for i = 1 to n
15                exchange a_ki with a_k'i
16          for i = k + 1 to n
17                a_ik = a_ik / a_k
18                for j = k + 1 to n
19                      a_ij = a_ij − a_ik a_kj
```

$\Theta(n^3)$

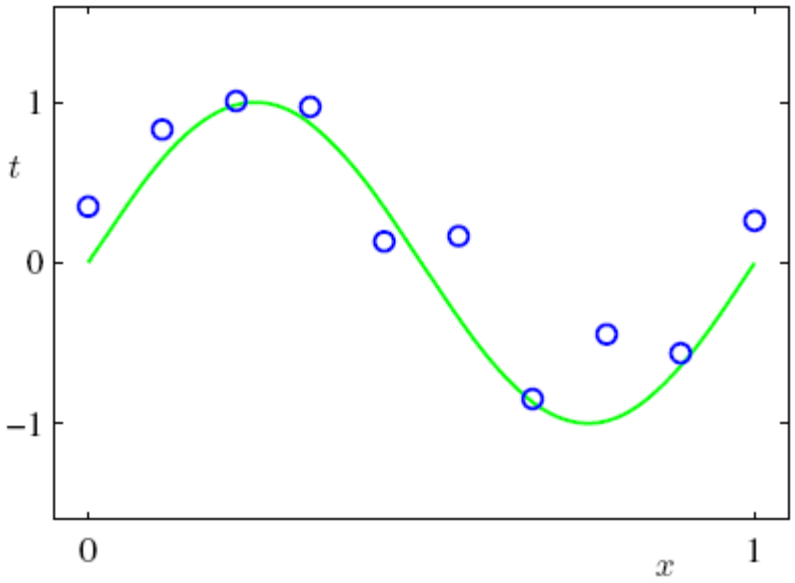
Computing a matrix inverse

- Suppose that we have an LUP decomposition of a matrix A in the form of three matrices L, U, and P such that $PA = LU$. The LUP decomposition of A can be computed in time $\Theta(n^3)$.
- Using LUP-SOLVE, we can solve an equation of the form $Ax = b$ in time $\Theta(n^2)$. Since the LUP decomposition depends on A but not b, we can solve another form $Ax = b'$ in additional time $\Theta(n^2)$.
- In general, one we have the LUP decomposition of A, we can solve, in time $\Theta(kn^2)$, k versions of the equation $Ax = b$ that differ only in b.

Computing a matrix inverse

- We can think of the equation $AX = I_n$, which defines the matrix X, the inverse of A, as a set of n distinct equations of form $Ax = b$.
- Let X_i denote the ith column of X, and recall the unit vector e_i is the ith column of I_n. We can then solve equation $AX = I_n$ for X by using LUP decomposition for A to solve each $AX_i = e_i$ separately for X_i.
- We can compute the inverse A^{-1} of a matrix A in time $\Theta(n^3)$.

Least-squares approximation



Suppose that we are given a set of m data points (x_1, y_1), (x_2, y_2), …, (x_m, y_m), where we know that the y_i are subject to measurement errors.

We would like to determine a function $F(x)$ such that the approximation errors

$E = F(x_i) - y_i$

are small for $i = 1, 2, \ldots, m$.

Least-squares approximation

Here, we assume that $F(x)$ has the form of a linearly weighted sum,

$$F(x_i) = \sum_{j=1}^{n} c_j f_j(x)$$

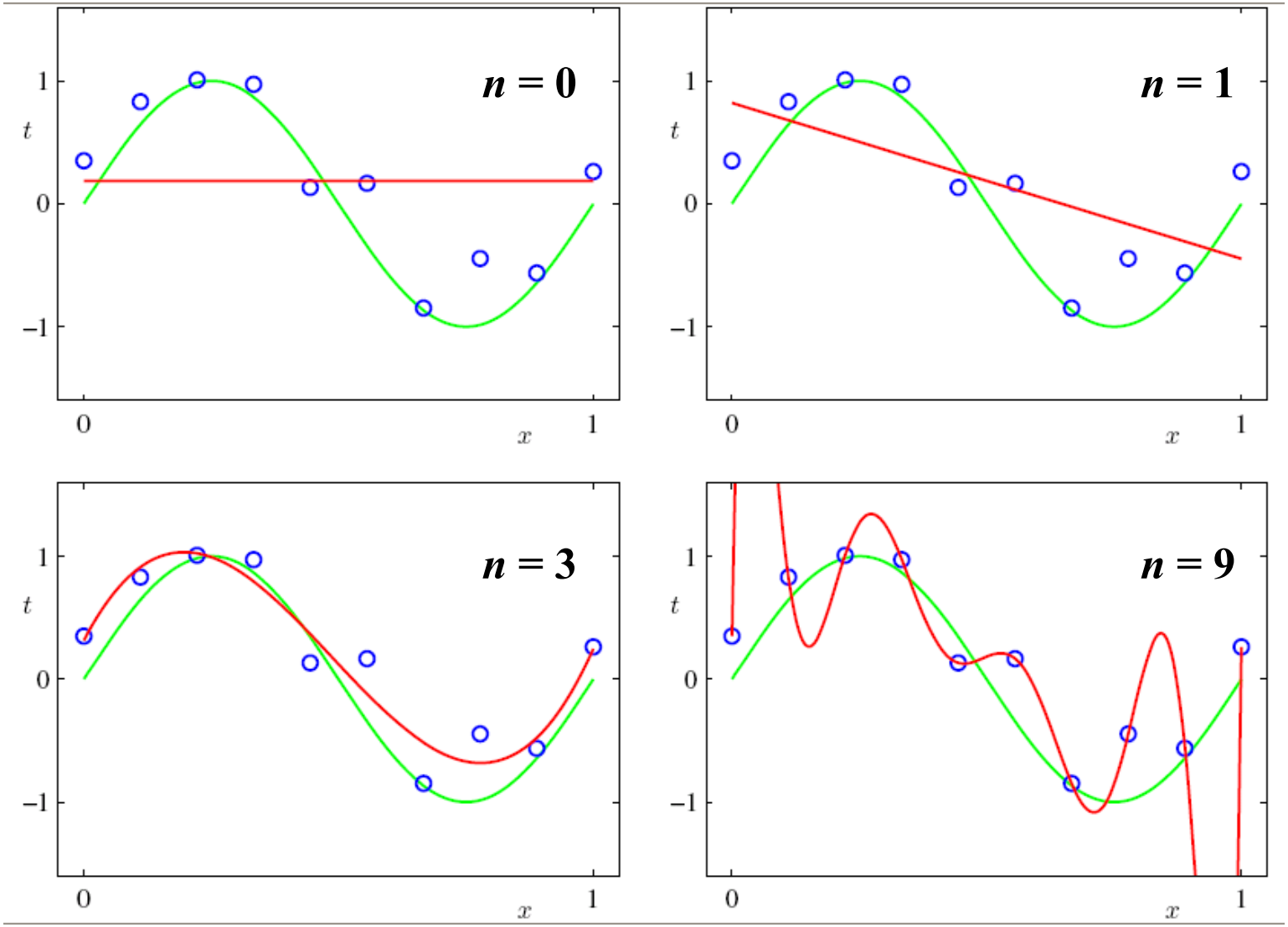
where the number of summands n and the specific ***basis functions*** f_j are chosen based on knowledge of the problem at hand.

A common choice is $f_j(x) = x^{j-1}$, which means that

$$F(x) = c_1 + c_2 x + c_3 x^2 + \cdots + c_n x^{n-1}$$

is a polynomial of degree $n - 1$ in x. Thus, given m data points, we which to calculate n coefficients $c_1, c_2, \ldots, c_n$ that minimize the approximation errors E.

Polynomials (various degrees)



Least-squares approximation

In any case, we choose a value of n that is less than m, we end up with an overdetermined set of equations whose solution we wish to approximate.

$$Ac = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_n) \end{pmatrix}$$

is the m-vector of "predicted values" for y, and c denotes the desired n-vector of coefficients.

Least-squares approximation

Thus, the m-vector of ***approximation errors*** is

$$E = Ac - y$$

To minimize approximation errors, we choose to minimize the norm of the error vector E, which gives us a ***least-squares solutions***.

$$\| E \| = \left(\sum_{i=1}^{m} E_i^2 \right)^{1/2} \text{ where } E_i = a_{ij} c_j - y_j$$

We can minimize $\|E\|$ by differentiating $\|E\|^2$ with respect to each c_k and them setting the result to 0:

$$\frac{d \| E \|^2}{dc_k} = \sum_{i=1}^{m} 2 \left(\sum_{j=1}^{n} a_{ij} c_j - y_i \right) a_{ik} = 0$$

Least-squares approximation

Then n equations for $k = 1, 2, \ldots, n$ are equivalent to the single matrix equation

$(Ac - y)^{\mathrm{T}} A = 0$ or, equivalently

$A^{\mathrm{T}}(Ac - y) = 0$

which implies

$A^{\mathrm{T}} Ac = A^{\mathrm{T}} y$

In statistics, this is called the ***normal equation***.

The matrix is symmetric and if A has full column rank, then the solution to equation is

$c = ((A^{\mathrm{T}} A)^{-1} A^{\mathrm{T}}) y = A^{+} y$

where the matrix A^{+} is the ***pseudoinverse*** of the matrix A.

Any question?

Xiaoqing Zheng
Fundan University